# Manual Testing Notes

## ➢ Manual Testing

Manual testing is a type of software testing where testers manually execute test cases without using any automation tools. The tester checks the application step-by-step from an end-user perspective.

### Key Points:

- No automation tools are used
- Focuses on user behavior and usability
- Suitable for exploratory and UI testing

### Example:

A tester opens a login page, enters username and password, and verifies if login works correctly.

## ➢ Functional Testing

Functional testing verifies that each function of the software works according to the specified requirements.

### Key Points:

- Based on requirements and specifications
- Checks input vs expected output
- Does not consider internal code logic

### Example:

Clicking the "Login" button should log the user into the system with valid credentials.

## ➢ Integration Testing

Integration testing checks whether different modules of the application work together correctly.

### Key Points:

- Focuses on interaction between modules
- Ensures data flow is correct
- Performed after unit testing

### Example:

Testing if login module correctly connects with the database and retrieves user data.

## ➢ Smoke Testing

Smoke testing is a quick testing process to check whether the basic functionalities of the application are working or not.

### Key Points:

- Performed on new builds
- Checks critical functionalities
- Decides whether to proceed with further testing

### Example:

Checking if application launches, login works, and dashboard loads successfully.

## ➢ Regression Testing

Regression testing ensures that new code changes do not negatively affect existing features.

### Key Points:

- Performed after changes or bug fixes
- Focuses on existing functionality
- Often automated

### Example:

After adding a new feature, re-testing login functionality to ensure it still works.

## ➢ Retesting

Retesting is done to verify that a previously failed test case now passes after the defect has been fixed.

### Key Points:

- Focuses on specific bug fixes
- Done after defect is resolved
- Uses same test data

### Example:

If login failed earlier due to a bug, tester checks login again after fix.

## ➢ Error vs Defect vs Bug

Error: A mistake made by developer during coding.

Defect: A deviation from expected behavior found during testing.

Bug: A defect reported to development team.

### Example:

Developer writes wrong formula (Error) → application gives wrong result (Defect) → tester reports it (Bug).

## ➢ Test Case

A test case is a set of conditions, inputs, steps, and expected results used to verify a particular functionality.

## ➢ Test Case Design Techniques

### 1. Boundary Value Analysis (BVA)

Boundary Value Analysis focuses on testing the values at the boundaries (edges) of input ranges, as defects are most likely to occur at these points.

**Key Idea:**
Test minimum, maximum, just below, and just above values.

**Example:**
If a field accepts values from **18 to 60**, test cases would be:

- 17 → Invalid (Below minimum)
- 18 → Valid (Minimum boundary)
- 19 → Valid (Just above minimum)
- 59 → Valid (Just below maximum)
- 60 → Valid (Maximum boundary)
- 61 → Invalid (Above maximum)

### 2. Equivalence Partitioning (EP)

Equivalence Partitioning divides input data into valid and invalid partitions, where test cases are designed to cover one value from each partition.

**Key Idea:**
Reduce number of test cases while maintaining coverage.

**Example:**
For input range **18–60**, partitions are:

- Less than 18 → Invalid
- 18 to 60 → Valid
- Greater than 60 → Invalid

Test cases:

- 10 → Invalid
- 30 → Valid
- 70 → Invalid

### 3. Decision Table Testing

Decision Table Testing is used when multiple conditions determine different outcomes.

**Example: Login Functionality**

| Username | Password | Expected Result |
|---|---|---|
| Valid | Valid | Login Success |
| Valid | Invalid | Error Message |
| Invalid | Valid | Error Message |
| Invalid | Invalid | Error Message |

**Key Benefit:**
Ensures all possible combinations are tested.

### 4. Error Guessing

Error Guessing is based on tester experience to identify areas where defects are likely.

**Examples:**

- Empty input fields
- Special characters
- Large input values
- Invalid formats

## ➢ Test Script

A test script is a set of instructions written in a programming language to automate test cases.

### Example:

Using Selenium to automate login functionality.

## Defect Life Cycle (Bug Life Cycle)

The **Defect Life Cycle** is the process through which a defect (bug) goes from identification to closure during software testing.

It ensures that defects are properly tracked, fixed, verified, and closed.

**Defect Life Cycle Stages:**

**1. New**
- When a tester identifies a defect, it is logged in the defect tracking tool.
- Initial status is **New**.

**2. Assigned**
- The defect is assigned to a developer by the Test Lead or Manager.
- Developer reviews the defect.

**3. Open**
- Developer starts working on the defect.
- The defect is now actively being fixed.

**4. Fixed**
- Developer fixes the defect and marks it as **Fixed**.
- The fix is deployed to the testing environment.

**5. Retest**
- Tester retests the defect using the same steps.
- Checks whether the issue is resolved.

**6. Verified**
- If the defect is fixed correctly, it is marked as **Verified**.

**7. Closed**
- After successful verification, the defect is marked as **Closed**.
- No further action is required.

**Other Important States**

**Rejected**
- Developer rejects the defect if:
    - It is not a bug
    - It is working as expected

**Duplicate**
- Same defect already exists in the system.

**Deferred**
- Fix is postponed to a future release.

**Not Reproducible**
- Developer/tester is unable to reproduce the issue.

**Reopened**
- If the defect still exists after fixing, tester reopens it.

## Defect Life Cycle Flow

New → Assigned → Open → Fixed → Retest → Verified → Closed

↓

Reopened

**Example (Real-Time):**

**Scenario:** Login button not working

1. Tester finds issue → marks as **New**
2. Assigned to developer → **Assigned**
3. Developer starts fixing → **Open**
4. Bug fixed → **Fixed**
5. Tester checks again → **Retest**
6. Works fine → **Verified**
7. Final closure → **Closed**